Puente: de los splines a los kernels

Repaso: funciones de densidad y distribucion

Proceso aleatorio generador de datos

Estimacion kernel de la densidad (KDE)

Kernel bivariado: hacia dimensiones mayores

Metodos kernel: regresion local

El kernel como medida de similitud

Sintesis y cierre

Referencias

Análisis Avanzado de Datos

Sesión 4: Métodos de Kernel

Nicolás López

Primer semestre de 2026

Puente: de los splines a los kernels

Repaso: funciones de densidad y distribucion

Proceso aleatorio generador de datos

Estimacion kernel de la densidad (KDE)

Kernel bivariado: hacia dimensiones mayores

Metodos kernel: regresion local

El kernel como medida de similitud

Sintesis y cierre

Referencias

1. Puente: de los splines a los kernels
2. Repaso: funciones de densidad y distribucion
3. Proceso aleatorio generador de datos
4. Estimacion kernel de la densidad (KDE)
5. Kernel bivariado: hacia dimensiones mayores
6. Metodos kernel: regresion local
7. El kernel como medida de similitud
8. Sintesis y cierre
9. Referencias

Puente: de los splines a los kernels

Repaso: funciones de densidad y distribucion

Proceso aleatorio generador de datos

Estimacion kernel de la densidad (KDE)

Kernel bivariado: hacia dimensiones mayores

Metodos kernel: regresion local

El kernel como medida de similitud

Sintesis y cierre

Referencias

Puente: de los splines a los kernels

Recordando las sesiones anteriores

Sesion 1 – Regularizacion: Ridge y Lasso controlan la magnitud de β. Siguen siendo modelos lineales **globales**.

Sesion 2 – Validacion: El hiperparametro de complejidad se selecciona minimizando ECM_{Test} por validacion cruzada.

Sesion 3 – Splines: Se rompe la linealidad con bases de funciones a trozos. El suavizamiento spline penaliza la rugosidad de f con $\lambda \int f''(x)^2 dx$.

El hilo comun

En todas las sesiones: un hiperparametro de complejidad (λ, df) controla el balance sesgo–varianza y se selecciona por validacion cruzada.

La idea comun: localidad

En splines, el ajuste en un punto depende de los datos de toda la region cercana al knot.

Esta sesion lleva la localidad al extremo:

> *Se estima f* ***en cada punto*** *x usando solo las observaciones* ***vecinas*** *de x, ponderadas por su cercania mediante una funcion K llamada* ***kernel****.*

Esta logica aplica para dos problemas:

- **Sin variable respuesta:** estimar la densidad $f(x)$ de los datos (KDE).
- **Con variable respuesta:** estimar $E(Y \mid X = x)$ (regresion local).

Puente: de los splines a los kernels

Repaso: funciones de densidad y distribucion

Proceso aleatorio generador de datos

Estimacion kernel de la densidad (KDE)

Kernel bivariado: hacia dimensiones mayores

Metodos kernel: regresion local

El kernel como medida de similitud

Sintesis y cierre

Referencias

Repaso: funciones de densidad y distribucion

Experimento aleatorio y espacio de probabilidad

- **Experimento aleatorio:** proceso con resultado incierto.
- **Espacio de probabilidad** $(\Omega, \mathcal{F}, P)$: Ω es el conjunto de posibles resultados; $\mathcal{F}$ los eventos a los que asignamos probabilidad; $P : \mathcal{F} \to [0, 1]$ satisface:
  - $P(A) \geq 0$ para todo $A \in \mathcal{F}$.
  - $P(\Omega) = 1$.
  - Para eventos disjuntos: $P(\cup_i A_i) = \sum_i P(A_i)$.

P esta definida sobre **eventos**. En la practica nos interesan **numeros**, no eventos: cuantos minutos espero, cuantos buses pasan, etc.

La variable aleatoria como mapeo

Para trabajar con numeros construimos: $X : \Omega \to \mathbb{R}$

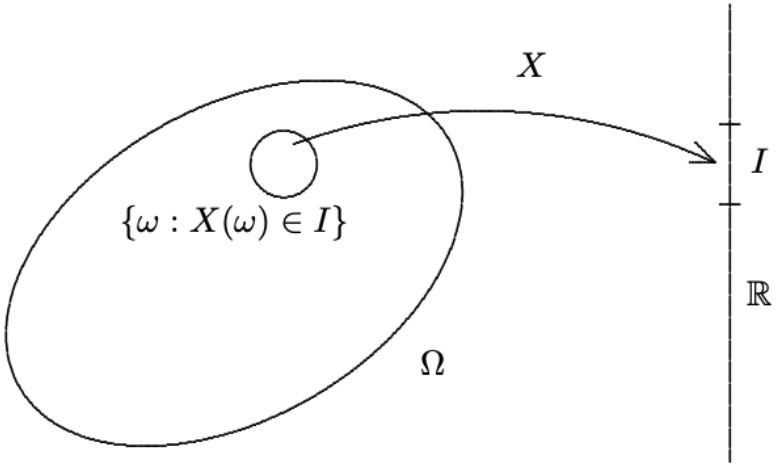


Figure 1: Tomado de: cimat.mx

Para todo intervalo $I \subset \mathbb{R}$, la preimagen $X^{-1}(I) = \{\omega \in \Omega : X(\omega) \in I\}$ es medible. Esto garantiza que podemos calcular probabilidades de eventos definidos en terminos de X.

La distribucion de X y la FDA

La medida inducida por X sobre $\mathbb{R}$:

$$P_X(I) = P(\{\omega \in \Omega : X(\omega) \in I\})$$

Tomando $I = (-\infty, x]$ obtenemos la **funcion de distribucion acumulada** (FDA):

$$F_X(x) = P(X \leq x) = P_X((-\infty, x])$$

La distribucion P_X queda completamente determinada por F_X.

Variable discreta: funcion masica de probabilidad

Si F_X tiene **saltos**, X es discreta:

$$P(X = x) = F_X(x) - F_X(x^-)$$

En estadistica parametrica se asume forma funcional indexada por θ:

- $P_\theta(x)$ – Modelo Bernoulli.
- $P_\theta(x)$ – Modelo Binomial.
- $P_\theta(x)$ – Modelo Poisson.
- $P_\theta(x)$ – Modelo Binomial negativo.
- $P_\theta(x)$ – Modelo Geometrico.

La estadistica **no parametrica** no impone familia alguna.

Puente: de los splines a los kernels

Repaso: funciones de densidad y distribucion

Proceso aleatorio generador de datos

Estimacion kernel de la densidad (KDE)

Kernel bivariado: hacia dimensiones mayores

Metodos kernel: regresion local

El kernel como medida de similitud

Sintesis y cierre

Referencias

La historia del paradero de autobuses

Escenario

Cada manana llego al paradero de mi barrio a esperar el bus. El bus tiene hora programada, pero a veces llega tarde, a veces temprano. Hay otros pasajeros esperando y varios buses durante el dia.

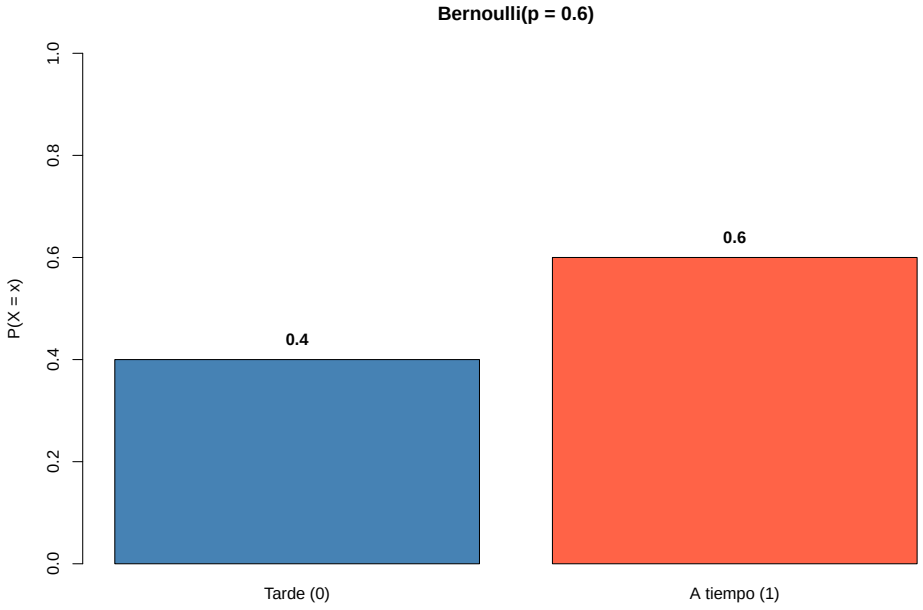
Este escenario simple genera naturalmente todas las distribuciones que veremos: conteos de buses, tiempos de espera, probabilidades de retraso.

Ejemplo: distribucion Bernoulli

Situacion: para cada bus que llega, me pregunto: ¿llega a tiempo? Si o no.

La probabilidad de que llegue a tiempo es $p = 0.6$.

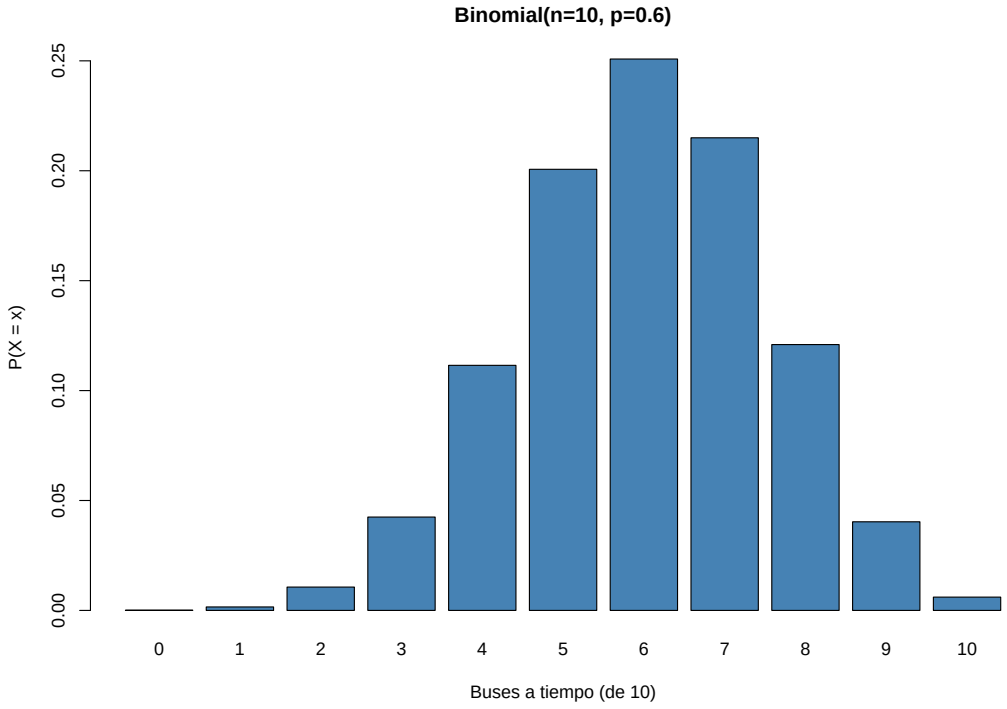
$X \sim \text{Ber}(p = 0.6)$: toma valores en $\{0, 1\}$.



Ejemplo: distribucion Binomial

Situacion: durante el dia pasan 10 buses. Cada uno llega a tiempo con probabilidad $p = 0.6$, de forma independiente. ¿Cuantos de los 10 buses llegan a tiempo?

$X \sim \text{Bin}(n = 10,\ p = 0.6)$.

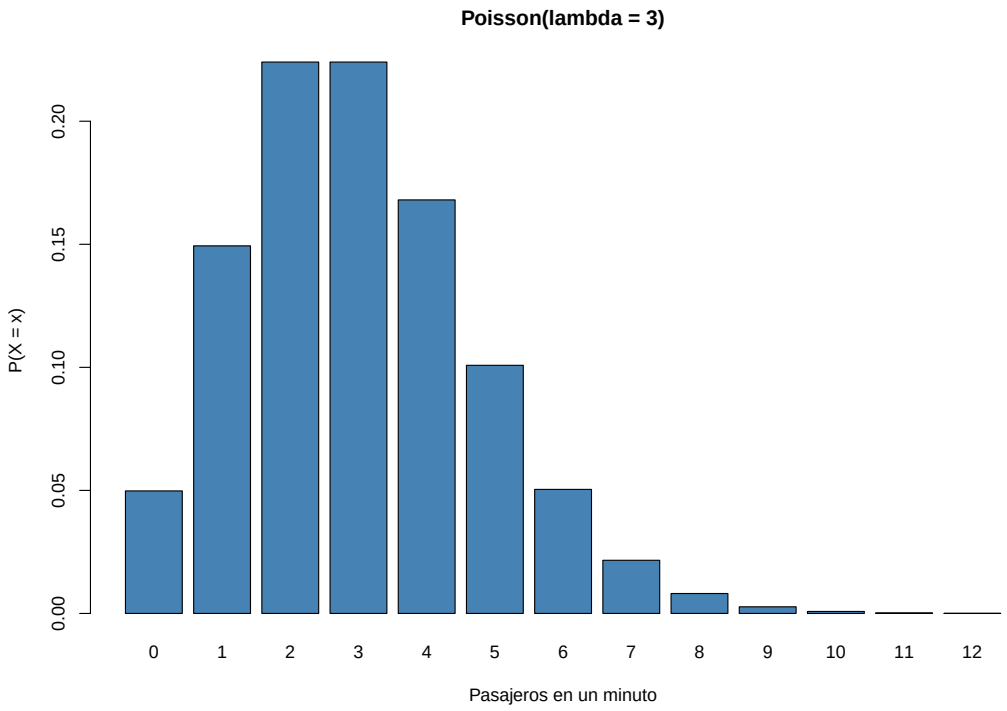


La moda esta en $x = 6$, coherente con $E(X) = np = 6$.

Ejemplo: distribucion Poisson

Situacion: en promedio llegan 3 pasajeros al paradero por minuto de forma aleatoria e independiente entre si. ¿Cuantos pasajeros llegan en el proximo minuto?

$X \sim \text{Poisson}(\lambda = 3)$.

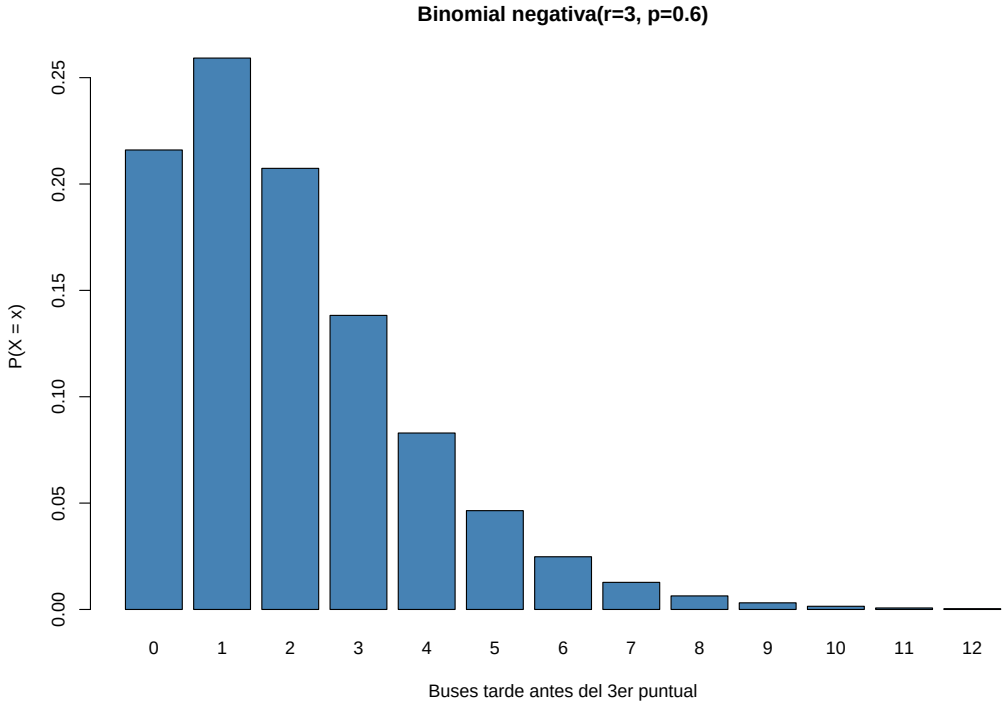


Util cuando se cuentan ocurrencias de eventos raros en un intervalo de tiempo o espacio fijo.

Ejemplo: distribucion Binomial negativa

Situacion: me propongo esperar hasta que 3 buses lleguen a tiempo. Cada bus tiene $p = 0.6$ de hacerlo. ¿Cuantos buses *retrasados* tendré que soportar antes de completar los 3 puntales?

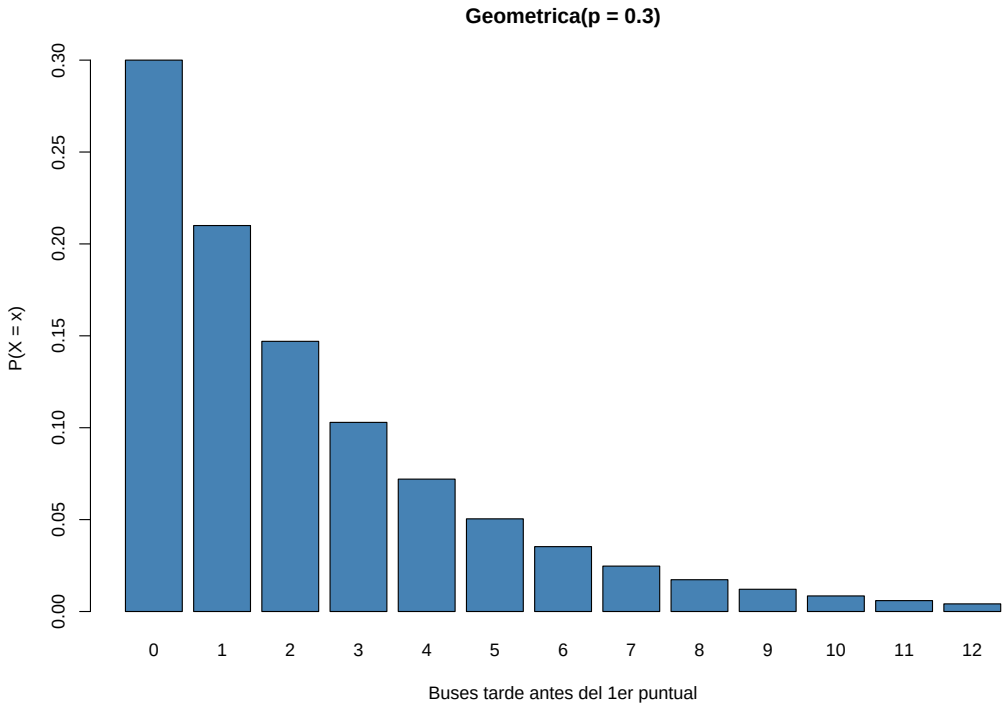
$X \sim \text{BinNeg}(r = 3,\ p = 0.6)$: numero de fracasos antes del r-esimo exito.



Ejemplo: distribucion Geometrica

Situacion: llego al paradero un dia lluvioso. Cada bus tiene probabilidad $p = 0.3$ de llegar a tiempo. ¿Cuantos buses retrasados pasan antes de que llegue el primero puntual?

$X \sim \text{Geom}(p = 0.3)$: numero de fracasos antes del primer exito.



Con p pequeno, es frecuente esperar varios buses.

Variable continua: funcion de densidad

Si F_X es continua, existe $f_X \geq 0$:

$$F_X(x) = \int_{-\infty}^{x} f_X(t)\, dt$$

Se caracteriza la aleatoriedad mediante la **funcion de densidad de probabilidad** (fdp).

En estadistica parametrica:

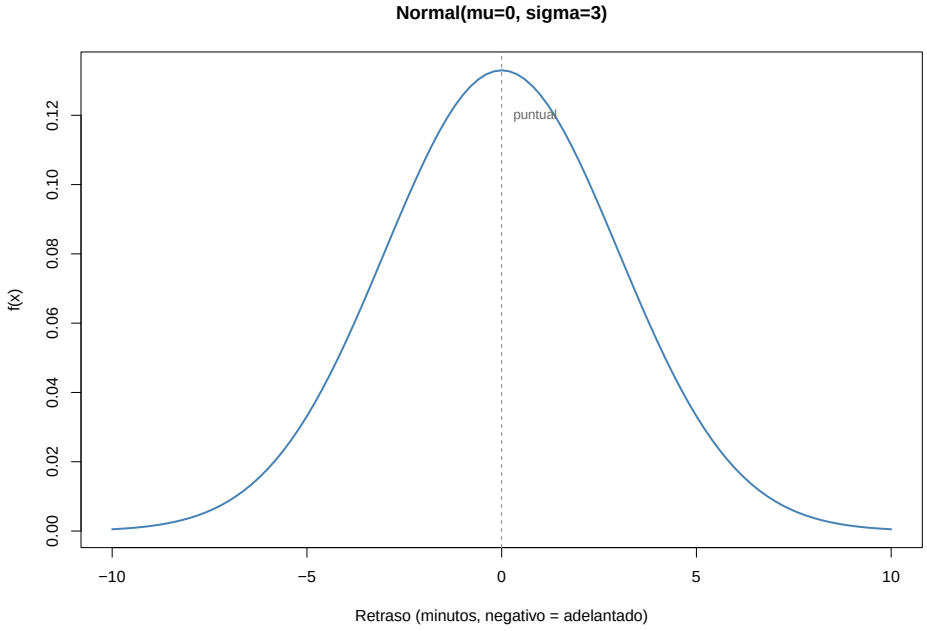
- $f_\theta(x)$ – Distribucion Normal.
- $f_\theta(x)$ – Distribucion Gamma.
- $f_\theta(x)$ – Distribucion t de Student.
- $f_\theta(x)$ – Distribucion F.

La estadistica **no parametrica** no impone familia alguna.

Ejemplo: distribucion Normal

Situacion: el bus tiene hora programada de llegada. El retraso real (en minutos) varia de forma simetrica alrededor de cero: a veces llega 5 min tarde, a veces 5 min antes, con mayor frecuencia puntual.

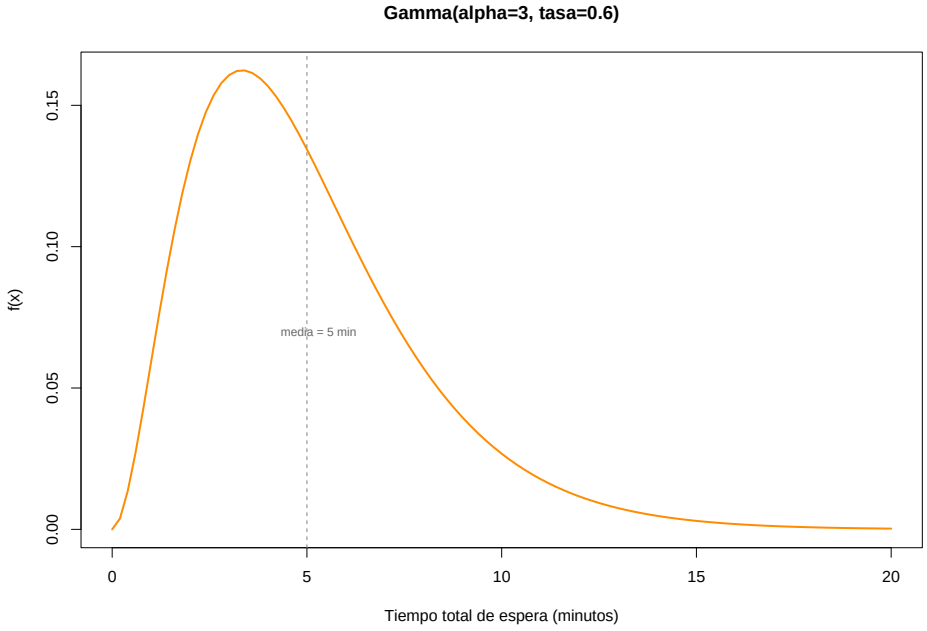
$X \sim N(\mu = 0,\ \sigma = 3)$: retraso en minutos.



Ejemplo: distribucion Gamma

Situacion: el tiempo total que paso esperando en el paradero hasta que lleguen 3 buses (sin importar si son puntuales o no). El tiempo entre buses es de 5 min en promedio.

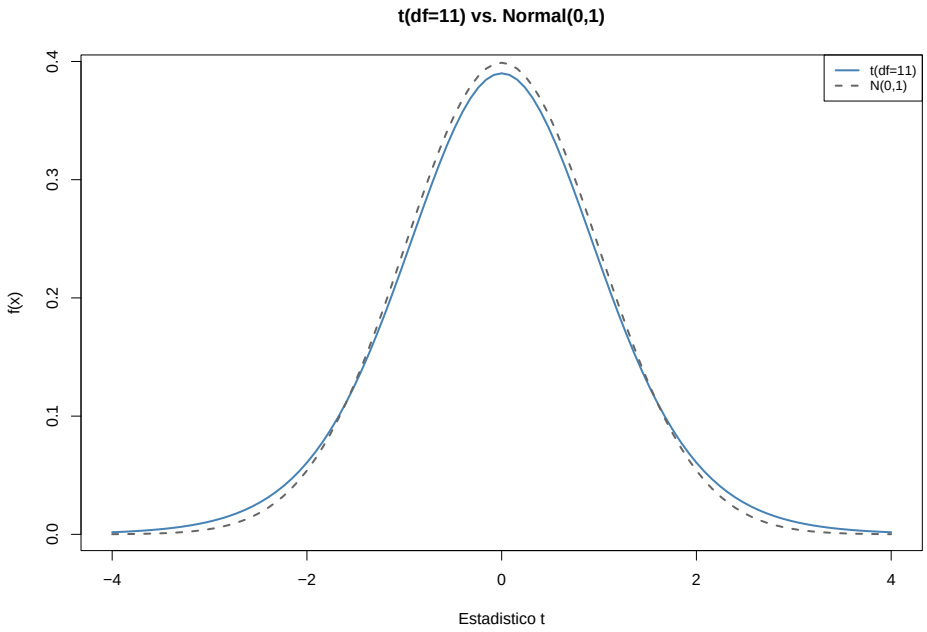
$X \sim \text{Gamma}(\alpha = 3,\ \text{tasa} = 0.6)$: suma de 3 tiempos exponenciales de media $1/0.6$.



Ejemplo: distribucion t de Student

Situacion: quiero probar si el retraso promedio del bus es cero ($H_0 : \mu = 0$) con solo $n = 12$ observaciones. Con pocos datos, el estadistico de prueba $t = \bar{X}/(\hat{\sigma}/\sqrt{n})$ sigue una distribucion t bajo H_0.

$T \sim t(\nu = 11)$ vs. $Z \sim N(0, 1)$.

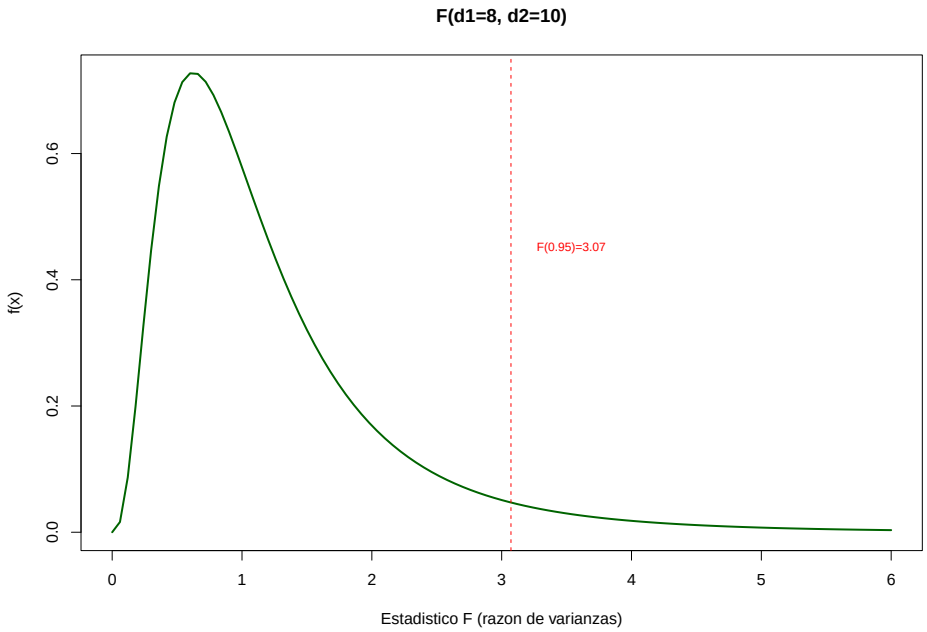


Las colas mas pesadas de la t reflejan la incertidumbre adicional por estimar σ con pocos datos.

Ejemplo: distribucion F

Situacion: comparo la variabilidad del retraso entre dos rutas: ruta A (matutina) y ruta B (nocturna). El estadistico $F = S_A^2/S_B^2$ sigue una distribucion F bajo $H_0 : \sigma_A^2 = \sigma_B^2$.

$F \sim F(d_1 = 8,\ d_2 = 10)$.



Valores grandes de F sugieren que una ruta es considerablemente mas irregular que la otra.

Propiedades fundamentales

$$\sum_x P(X = x) = 1 \qquad \int_{-\infty}^{+\infty} f(x)\, dx = 1$$

$$P(X = x) \geq 0 \quad \text{y} \quad f(x) \geq 0 \text{ para todo } x$$

Nota: $P(x) \leq 1$ siempre, pero $f(x)$ puede superar 1. Por ejemplo, el retraso del bus en un sistema muy preciso podria tener $f(0) = 10$ si los retrasos se concentran en $[0, 0.1]$ minutos: lo que importa es que el area total sea 1.

Conclusiones del repaso

(1) La v.a. X mapea resultados del experimento a numeros reales.

(2) La aleatoriedad de X se caracteriza por la fmp (discreta) o la fdp (continua), parametrica o no.

(3) Al escribir $X \sim f(x)$, asumimos que existe una ley que rige los datos. La estadistica no parametrica **no especifica esa ley**: la estima directamente desde las observaciones.

Puente: de los splines a los kernels

Repaso: funciones de densidad y distribucion

Proceso aleatorio generador de datos

Estimacion kernel de la densidad (KDE)

Kernel bivariado: hacia dimensiones mayores

Metodos kernel: regresion local

El kernel como medida de similitud

Sintesis y cierre

Referencias

Proceso aleatorio generador de datos

Muestra aleatoria

En inferencia estadistica se asume:

$$X_1, X_2, \ldots, X_n \overset{iid}{\sim} f(x)$$

Nosotros observamos las **realizaciones** $x_1, x_2, \ldots, x_n$.

Perspectiva parametrica: $X_i \sim f_\theta(x)$ con θ desconocido. Se asume la forma de f.

Perspectiva no parametrica: $X_i \sim f(x)$ sin asumir ninguna familia funcional.

Objetivo de esta sesion

Estimar f directamente desde los datos, sin asumir que pertenece a ninguna familia parametrica conocida. Esto es la **estimacion no parametrica de la densidad**.

Puente: de los splines a los kernels

Repaso: funciones de densidad y distribucion

Proceso aleatorio generador de datos

Estimacion kernel de la densidad (KDE)

Kernel bivariado: hacia dimensiones mayores

Metodos kernel: regresion local

El kernel como medida de similitud

Sintesis y cierre

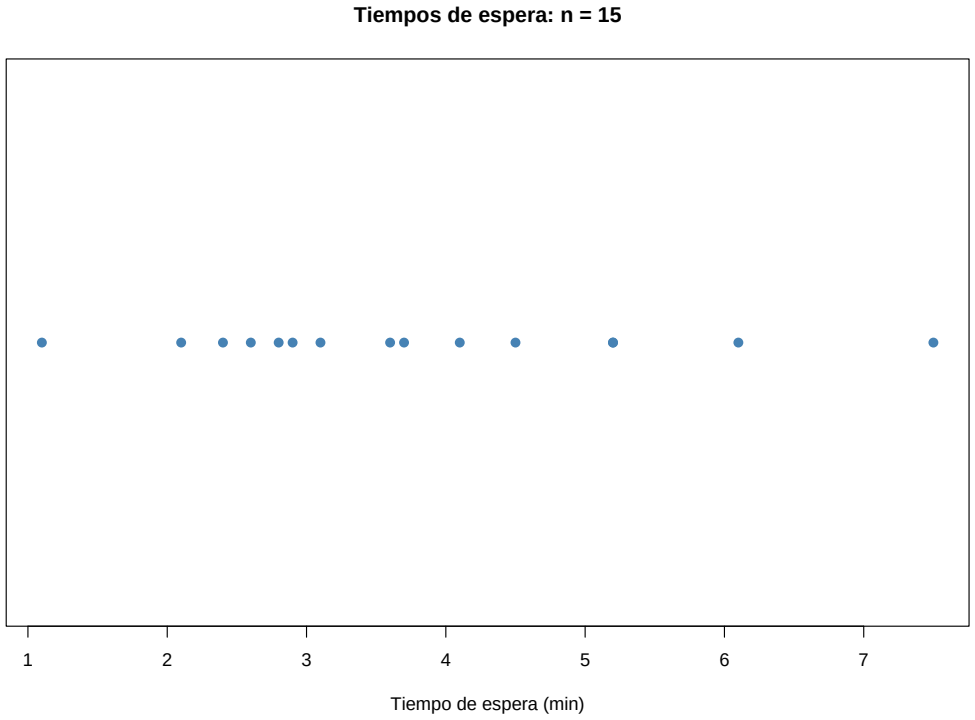
Referencias

Estimacion kernel de la densidad (KDE)

El problema: datos sin modelo

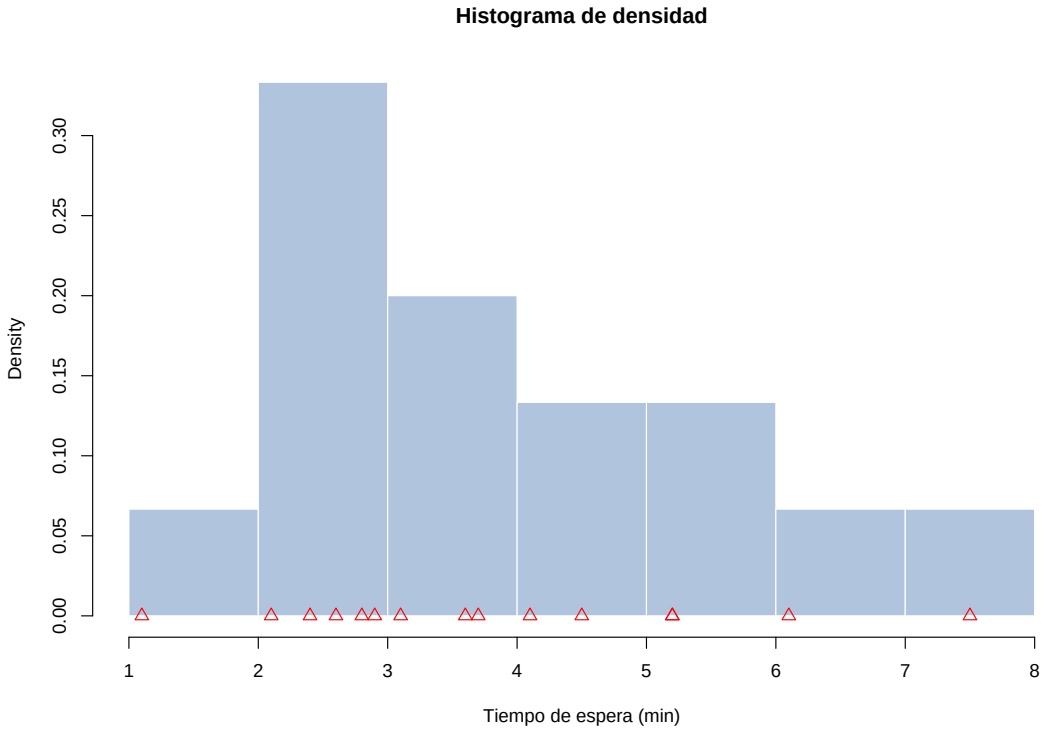
Tenemos $n = 15$ mediciones del tiempo de espera del bus (en minutos) en distintos dias. ¿Como estimamos la distribucion subyacente $f(x)$ sin asumir que es Normal u otra familia?

```
xval <- c(1.1,2.1,2.4,2.6,2.8,2.9,
          3.1,3.6,3.7,4.1,4.5,5.2,5.2,6.1,7.5)
plot(xval, y=rep(0, length(xval)),
     yaxt="n", ylab="", xlab="Tiempo de espera (min)",
     main="Tiempos de espera: n = 15",
     pch=16, col="steelblue", cex=1.2)
```



El histograma como estimador de densidad

```
h_dens <- hist(xval, prob=TRUE,
               main="Histograma de densidad",
               xlab="Tiempo de espera (min)",
               col="lightsteelblue", border="white")
points(xval, y=rep(0, length(xval)),
       pch=2, col="red", cex=1.2)
```



El eje y es densidad: el area de cada barra es la probabilidad del intervalo correspondiente.

El histograma cumple ser una fdp

```r
cat("Breaks:", h_dens$breaks, "\n")
```

```
## Breaks: 1 2 3 4 5 6 7 8
```

```r
cat("Densidades:", round(h_dens$density, 4), "\n")
```

```
## Densidades: 0.0667 0.3333 0.2 0.1333 0.1333 0.0667 0.0667
```

```r
l_base   <- diff(h_dens$breaks)
l_altura <- h_dens$density
cat("Area total:", sum(l_base * l_altura), "\n")
```

```
## Area total: 1
```

El area es 1 y la densidad es no negativa: las condiciones de una fdp valida se satisfacen.

Que significa "hay una v.a. real asociada X"

El resultado `area = 1` confirma que los datos se comportan como si vinieran de una variable aleatoria X con fdp bien definida sobre $\mathbb{R}$.

No afirmamos que los tiempos de espera sigan una familia particular. Solo decimos: existe una distribucion desconocida f tal que $X \sim f(x)$, y las observaciones son realizaciones de ese proceso.

La formula del histograma

La pregunta estadistica es: ¿como estimar f sin imponer una forma? El histograma es la primera respuesta: cruda pero que ya satisface las propiedades de una fdp.

$$\hat{f}_h(x) = \frac{1}{nh} \sum_{i=1}^{n} I(x_i \in [b_{j-1}, b_j])$$

donde $[b_{j-1}, b_j]$ es el intervalo de ancho h que contiene a x.

Problemas:

(1) Escalonada: los tiempos de espera no cambian abruptamente en un punto preciso.

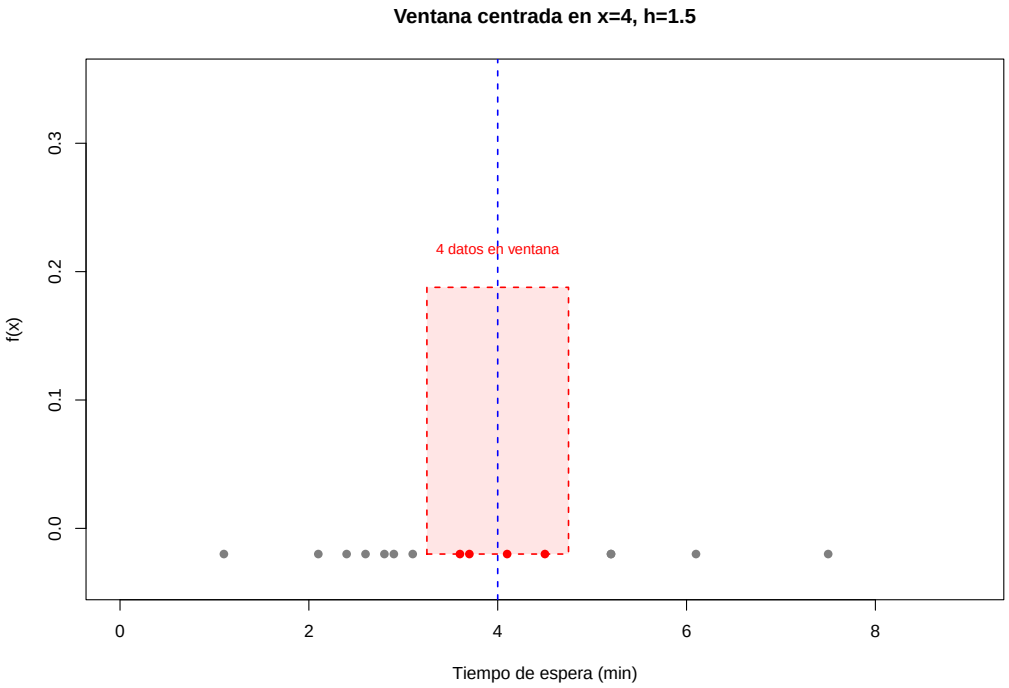
(2) Breaks arbitrarios: cambiar la posicion inicial de los bins cambia la forma del histograma.

(3) Densidad constante en cada bin: aunque dentro del bin haya variacion, todos los puntos reciben la misma altura.

Solucion: centrar la ventana de estimacion en cada punto x donde se quiere estimar f.

Histograma centrado: motivacion

En lugar de dividir el eje en bins fijos, para estimar f en el punto x contamos cuantas observaciones caen en la ventana $[x - h/2,\ x + h/2]$:



Contamos cuantos x_i caen en $[x - h/2,\ x + h/2]$ y dividimos por $n \cdot h$:

$$\hat{f}_h(x) = \frac{1}{nh} \sum_{i=1}^{n} I\left(x_i \in \left[x - \tfrac{h}{2},\ x + \tfrac{h}{2}\right]\right)$$

Podemos reexpresar la indicadora

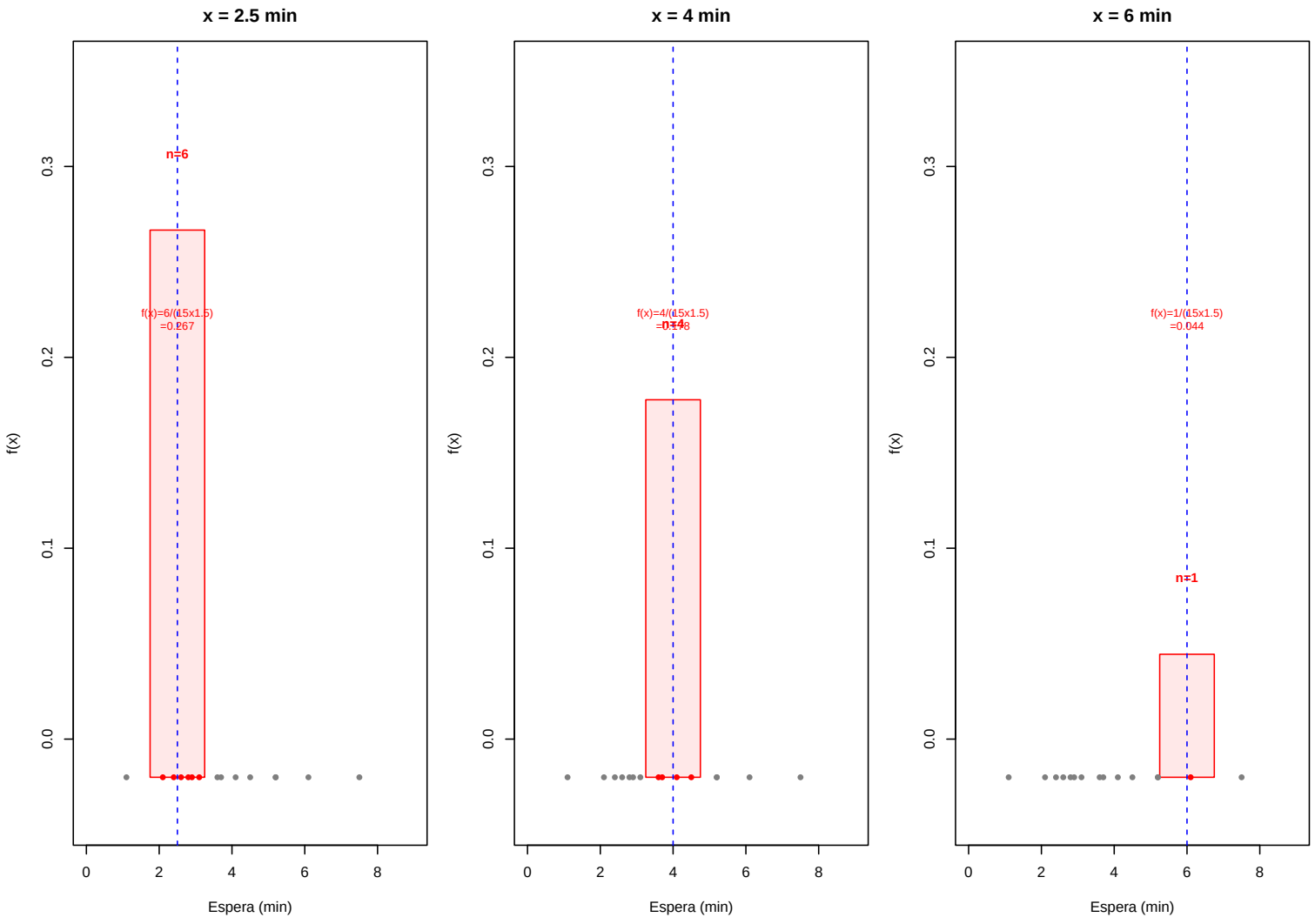
Paso 1 – reescribir la condicion: $x_i \in [x - h/2,\ x + h/2]$ equivale a $(x - x_i) \in [-h/2,\ h/2]$, que equivale a:

$$\frac{x - x_i}{h} \in \left[-\tfrac{1}{2},\ +\tfrac{1}{2}\right]$$

Paso 2 – interpretacion: la cantidad $|x - x_i|/h$ mide la distancia entre x y la observacion x_i **en unidades del ancho de banda** h. Datos con $|x - x_i|/h < 1/2$ son "vecinos" de x; los demas no influyen en la estimacion.

Histograma centrado: la ventana se desliza

La estimacion se calcula para cada x: la ventana se mueve a lo largo del dominio.



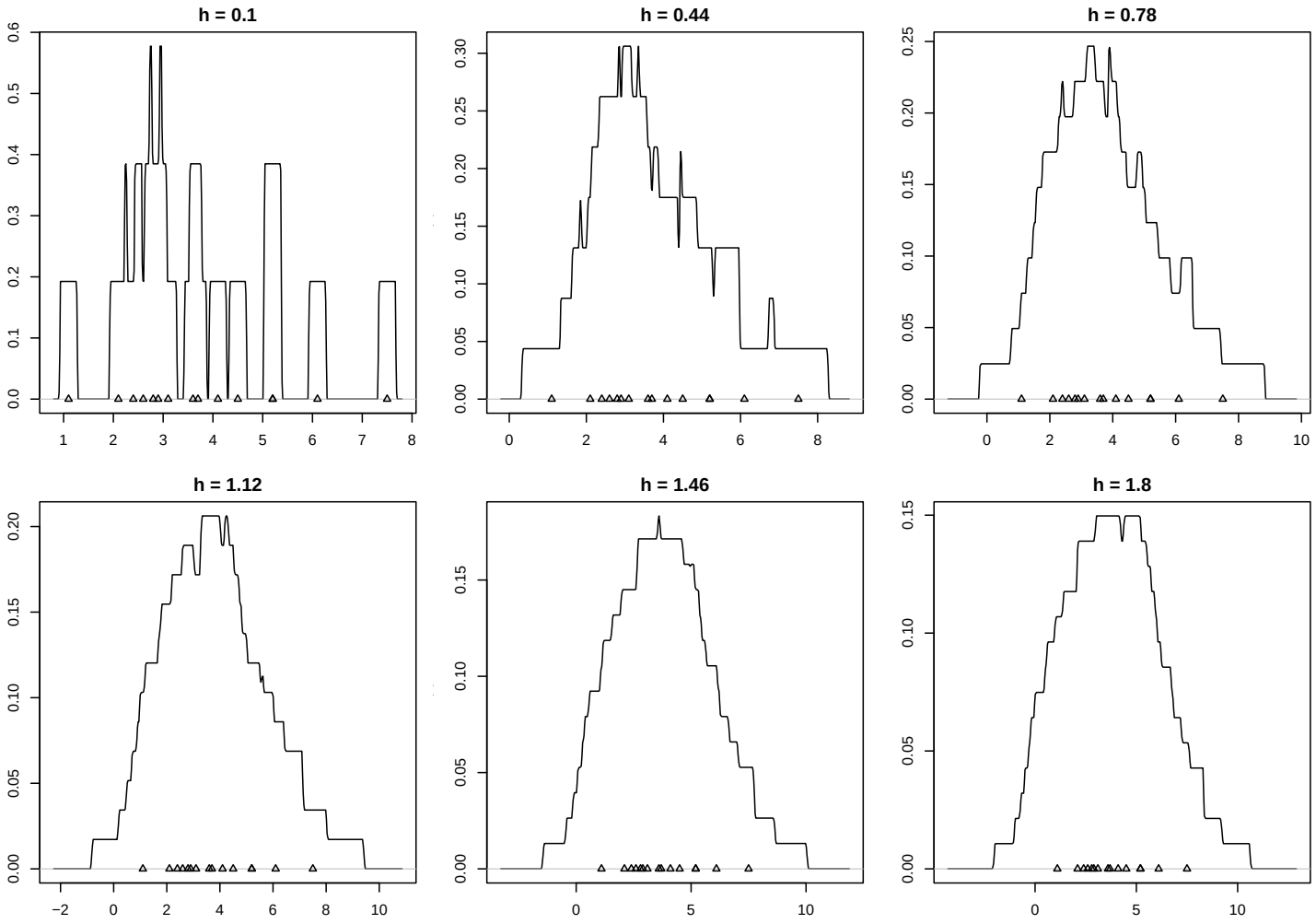
Del histograma centrado al kernel

El histograma centrado puede escribirse como:

$$\hat{f}_h(x) = \frac{1}{nh}\sum_{i=1}^{n} K\left(\frac{x - x_i}{h}\right) = \frac{1}{nh}\sum_{i=1}^{n} I\left(\frac{x - x_i}{h} \in [-1/2, +1/2]\right)$$

con $K(t) = I(t \in [-1/2, +1/2])$ (cómo se dibuja?): el **kernel rectangular**, que da peso 1 a los datos dentro de la ventana y 0 a los demas.

Efecto del ancho de banda h



Generalizacion: en lugar de un peso uniforme, usar pesos que **decrezcan suavemente** con la distancia. Eso es un kernel distinto del rectangular.

Una funcion K es kernel valido si:

- $K(t) \geq 0$, $K(-t) = K(t)$,
- $\int K(t)dt = 1$ y $K(t) \to 0$ para $|t| \to \infty$.

Por ejemplo, el *kernel gausiano* cumple estas condiciones.

Trade-off sesgo-varianza en h

Al igual que λ en Ridge/Lasso y df en splines:

- h **pequeno:** pocos vecinos, estimacion muy ruidosa. **Alta varianza, bajo sesgo.**
- h **grande:** muchos vecinos, estimacion muy suave, puede perder la forma real de f. **Baja varianza, alto sesgo.**

$$ECM(\hat{f} \mid h) = \int V(\hat{f}(x|h))\, dx + \int B^2(\hat{f}(x|h))\, dx$$

Existe un h optimo. Para el kernel gaussiano:

$$h_o \approx 1.06\, \hat{\sigma}\, n^{-1/5}$$

Esta es la **regla de Silverman**, implementada en R por defecto (`bw = "nrd0"`).

Tipos de kernels

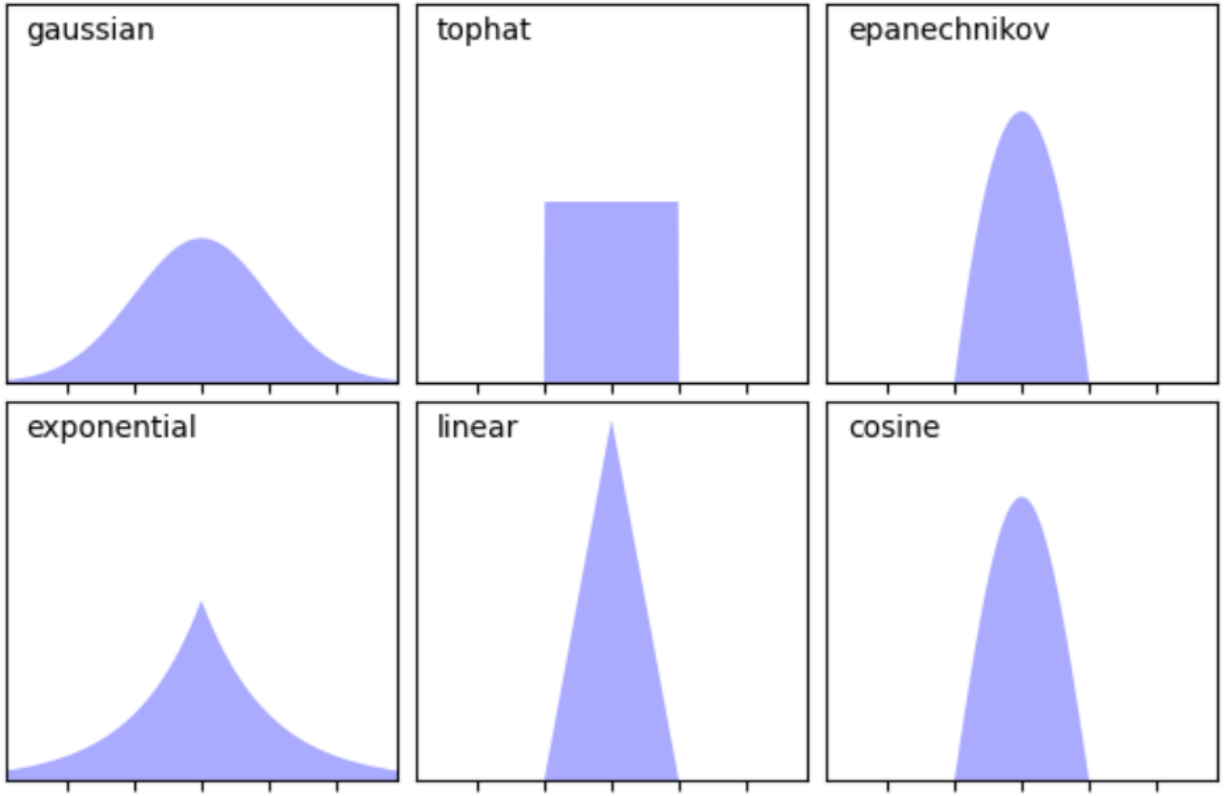


Figure 2: Tomado de: scikit-learn.org

Todos producen una $\hat{f}$ valida.

Puente: de los splines a los kernels

Repaso: funciones de densidad y distribucion

Proceso aleatorio generador de datos

Estimacion kernel de la densidad (KDE)

Kernel bivariado: hacia dimensiones mayores

Metodos kernel: regresion local

El kernel como medida de similitud

Sintesis y cierre

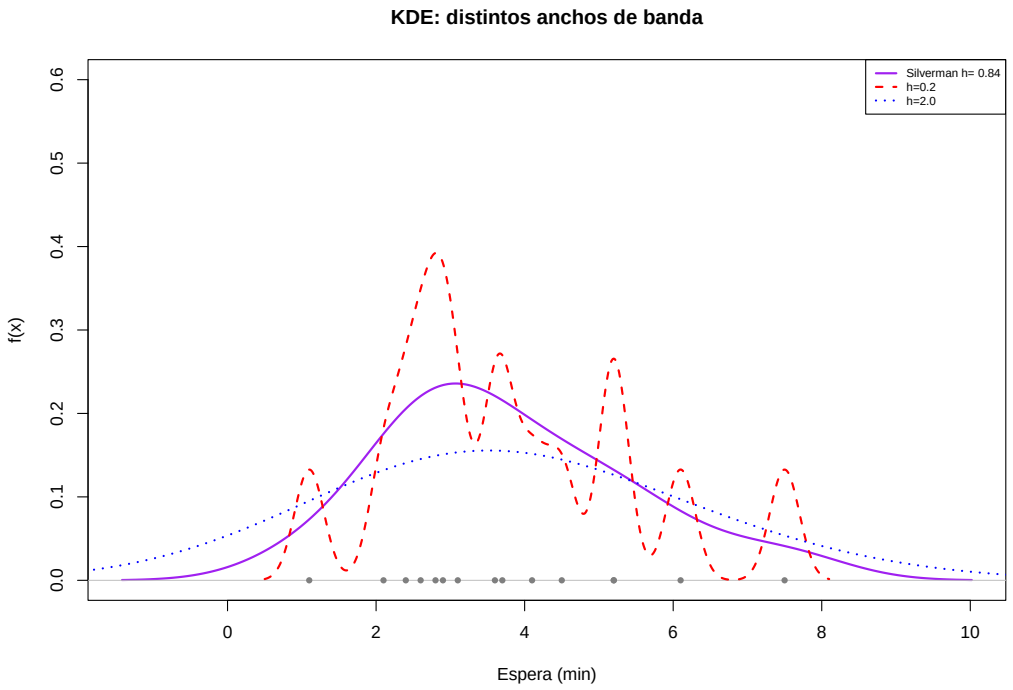
Referencias

El kernel gaussiano es el mas usado: da pesos que decaen suavemente con la distancia.

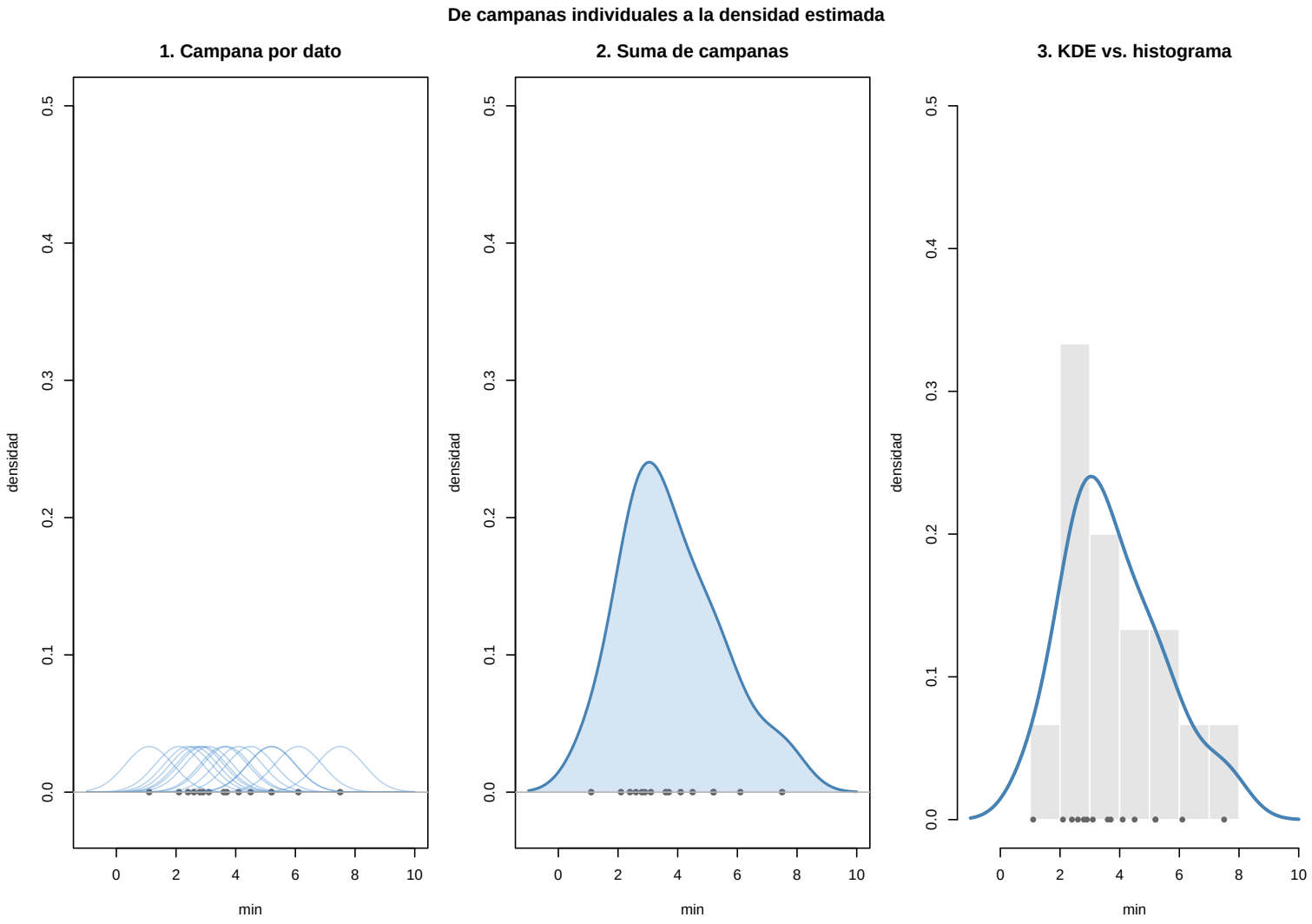
$$K(t) = \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}t^2}$$

KDE en R: density()

```
kde_opt  <- density(xval)
kde_ch   <- density(xval, bw=0.2)
kde_anch <- density(xval, bw=2.0)
plot(kde_opt, lwd=2, col="purple",
     main="KDE: distintos anchos de banda",
     xlab="Espera (min)", ylab="f(x)", ylim=c(0,0.6))
lines(kde_ch,   lwd=2, col="red",  lty=2)
lines(kde_anch, lwd=2, col="blue", lty=3)
points(xval, rep(0,length(xval)), pch=16,
       col="grey50", cex=0.8)
legend("topright", cex=0.65,
       legend=c(paste("Silverman h=",round(kde_opt$bw,2)),
                "h=0.2","h=2.0"),
       col=c("purple","red","blue"), lty=c(1,2,3), lwd=2)
```



Construccion paso a paso del KDE



Aplicaciones de la estimacion de densidad

Una vez se tiene $\hat{f}(x)$, se puede responder preguntas sin asumir un modelo parametrico:

1. **Probabilidad de un evento arbitrario:** $P(a \leq X \leq b) = \int_a^b \hat{f}(x)\, dx$, por ejemplo, la probabilidad de esperar mas de 6 minutos.

2. **Deteccion de esperas atipicas:** dias con $\hat{f}(x_i) < \varepsilon$ son improbables bajo la distribucion estimada del sistema.

3. **Comparacion entre rutas:** se estima $\hat{f}_1$ para la ruta A y $\hat{f}_2$ para la ruta B, y se comparan visualmente o con distancias entre densidades (por ejemplo, la distancia de Kullback-Leibler $\int f_1 \log(f_1/f_2)$).

4. **Simulacion de escenarios:** muestrear de $\hat{f}$ para simular tiempos de espera en situaciones donde no se tienen suficientes datos reales.

5. **Analisis exploratorio sin supuestos:** antes de ajustar cualquier modelo, visualizar si los datos son bimodales, asimetricos o tienen colas pesadas.

Puente: de los splines a los kernels

Repaso: funciones de densidad y distribucion

Proceso aleatorio generador de datos

Estimacion kernel de la densidad (KDE)

Kernel bivariado: hacia dimensiones mayores

Metodos kernel: regresion local

El kernel como medida de similitud

Sintesis y cierre

Referencias

Kernel bivariado: hacia dimensiones mayores

Extension a dos dimensiones

En 1D: la distancia es $|x - x_i|$. En 2D: $\mathbf{x} = (x_1, x_2)$ y se usa la norma euclidiana $||\mathbf{x} - \mathbf{x}_i||$.

El KDE bivariado es:

$$\hat{f}_h(\mathbf{x}) = \frac{1}{n} \sum_{i=1}^{n} K_h(\mathbf{x} - \mathbf{x}_i)$$

Kernel producto (con anchos de banda independientes):

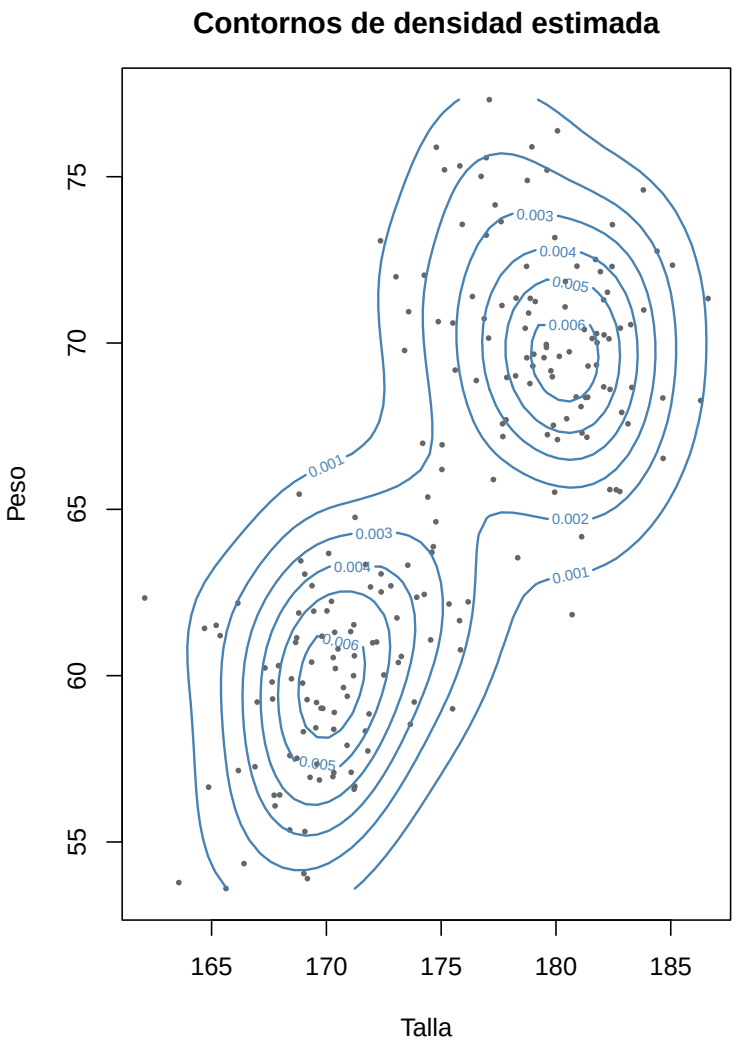
$$K_{h_1,h_2}(\mathbf{x} - \mathbf{x}_i) = \frac{1}{h_1} K\left(\frac{x_1 - x_{i1}}{h_1}\right) \cdot \frac{1}{h_2} K\left(\frac{x_2 - x_{i2}}{h_2}\right)$$

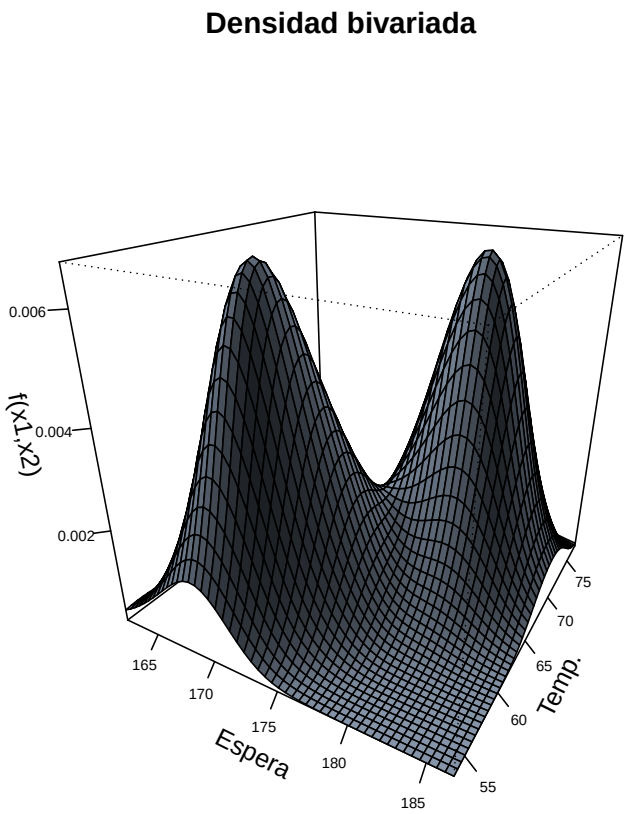
Kernel radial gaussiano (un solo h, distancia euclidiana):

$$K_h(\mathbf{x} - \mathbf{x}_i) = \frac{1}{2\pi h^2} \exp\left(-\frac{||\mathbf{x} - \mathbf{x}_i||^2}{2h^2}\right)$$

Si en 1D ponías una "campanita" sobre una línea, en 2D vas a poner una "montañita" (un domo) sobre un plano.

KDE bivariado en R: kde2d()





`kde2d()` usa por defecto el kernel radial gaussiano con h seleccionado por la regla de Silverman en 2D.

De 2D a dimensiones arbitrarias

En 1D: $K_h(x - x_i)$ usa $|x - x_i|/h$.

En 2D: $K_h(\mathbf{x} - \mathbf{x}_i)$ usa $||\mathbf{x} - \mathbf{x}_i||/h$.

En $\mathbb{R}^p$: la misma formula con la norma en p dimensiones.

Una pregunta interesante es: ¿que pasa cuando los datos **no son vectores numericos**? Por ejemplo:

- textos.
- moleculas.
- grafos de conexiones,
- imagenes.

Para esos casos necesitamos una medida de similitud mas general que la distancia euclidiana. Lo revisitaremos más adelante.

Puente: de los splines a los kernels

Repaso: funciones de densidad y distribucion

Proceso aleatorio generador de datos

Estimacion kernel de la densidad (KDE)

Kernel bivariado: hacia dimensiones mayores

Metodos kernel: regresion local

El kernel como medida de similitud

Sintesis y cierre

Referencias

Metodos kernel: regresion local

Del KDE a la regresion

KDE estima $f(x)$: la densidad de una sola variable, sin variable respuesta.

Regresion local: extiende la idea a estimar $E(Y \mid X = x)$, usando las observaciones cercanas a x ponderadas por el kernel.

Esto conecta con la Sesion 3: el suavizamiento spline tambien pondera observaciones, pero las ponderaciones vienen de la penalizacion de rugosidad $\int f''(x)^2 dx$, no de un kernel explicito.

El estimador de Nadaraya-Watson

Ajustar una **constante local** (grado 0) en cada x:

$$\hat{f}(x) = \underset{\alpha}{\operatorname{argmin}} \sum_{i=1}^{n} K\left(\frac{x - x_i}{h}\right)(y_i - \alpha)^2$$

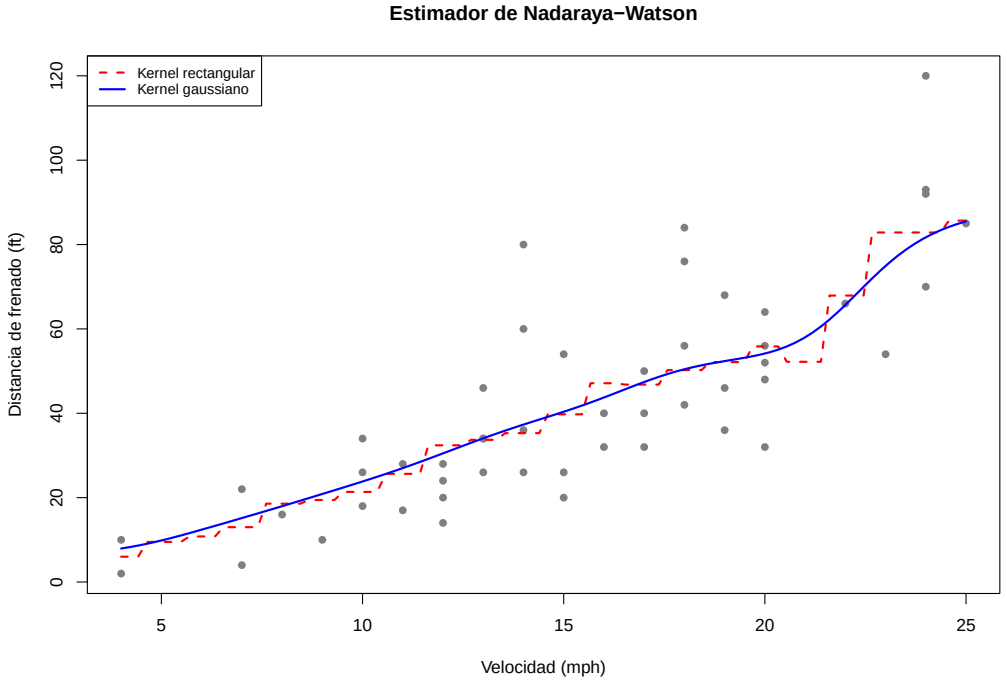
La solucion es el **promedio ponderado** de los y_i:

$$\hat{f}(x) = \frac{\sum_{i=1}^{n} K_h(x - x_i)\, y_i}{\sum_{i=1}^{n} K_h(x - x_i)}$$

Datos cercanos a x reciben mas peso; los lejanos casi no influyen. h determina el alcance.

Nadaraya-Watson en R

```
plot(cars$speed, cars$dist,
     xlab="Velocidad (mph)",
     ylab="Distancia de frenado (ft)",
     main="Estimador de Nadaraya-Watson",
     pch=16, col="grey50")
sm_box    <- ksmooth(cars$speed, cars$dist,
                     "box",    bandwidth=5)
sm_normal <- ksmooth(cars$speed, cars$dist,
                     "normal", bandwidth=5)
lines(sm_box,    col="red",  lwd=2, lty=2)
lines(sm_normal, col="blue", lwd=2)
legend("topleft", cex=0.8,
       legend=c("Kernel rectangular","Kernel gaussiano"),
       col=c("red","blue"), lty=c(2,1), lwd=2)
```



Regresion lineal local

Ajustar una **recta local** en cada x (grado 1):

$$\hat{f}(x) = \underset{\alpha,\beta_1}{\operatorname{argmin}} \sum_{i=1}^{n} K\left(\frac{x - x_i}{h}\right)(y_i - \alpha - \beta_1(x - x_i))^2$$

La prediccion en x es $\hat{\alpha}$. El termino $\beta_1(x - x_i)$ adapta el modelo a la pendiente local, **reduciendo el sesgo en los extremos del dominio**, donde Nadaraya-Watson tiende a subestimar o sobreestimar.

La imagen de la regresion local

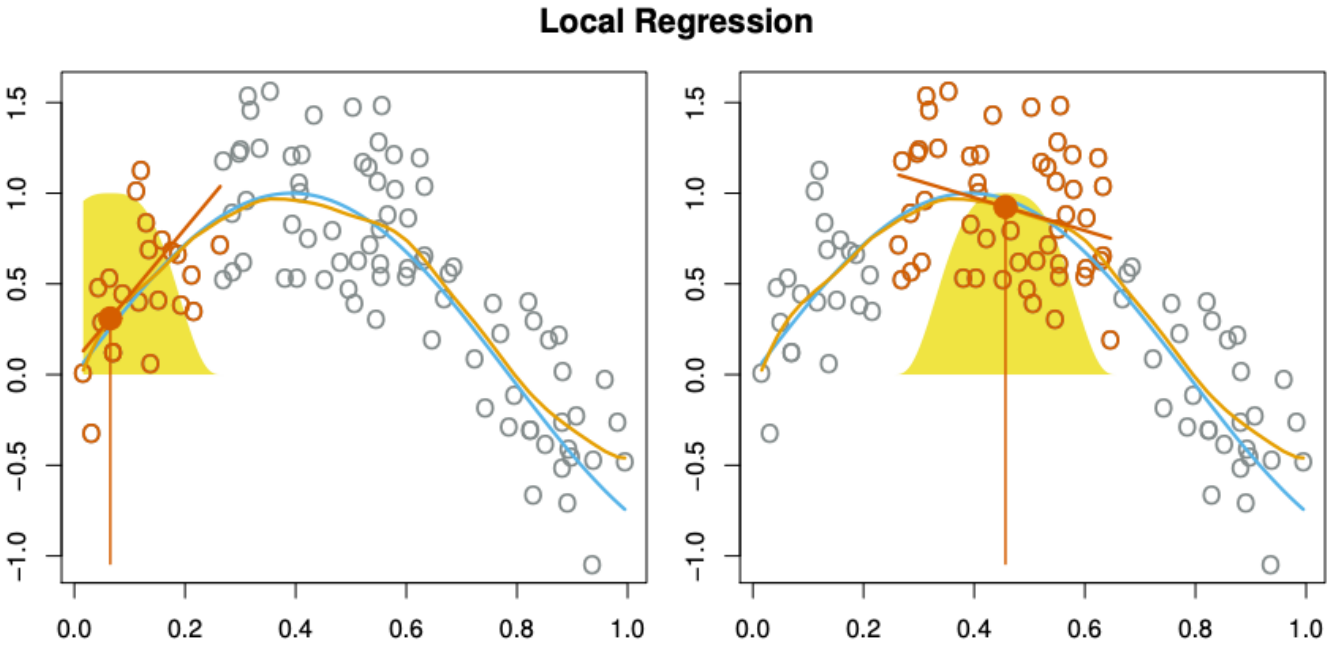


Figure 3: Hastie, Tibshirani, Friedman. ESL. En naranja: $\hat{f}$. En azul: f real.

La regresion lineal local corrige el sesgo en los extremos, visible en la diferencia entre las dos curvas.

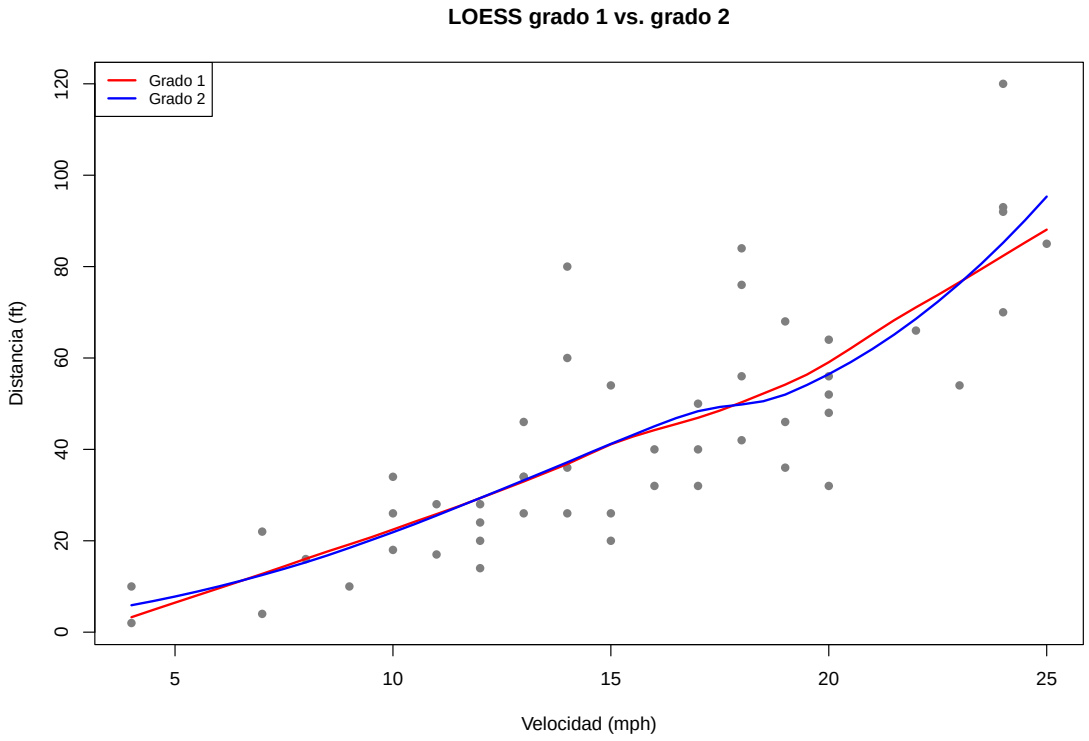
Regresion polinomial local (LOESS)

Generalizando a polinomios de grado p:

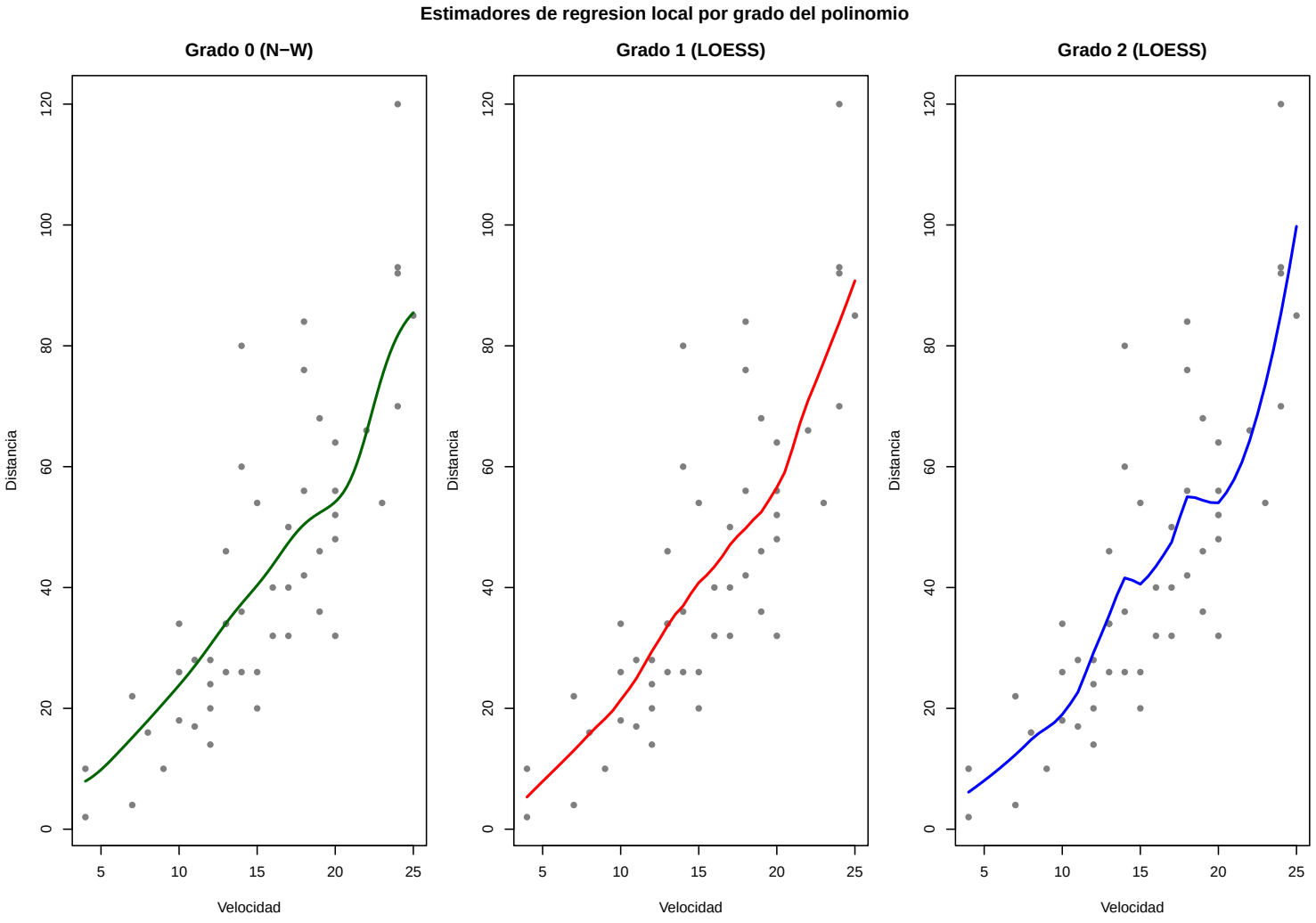
$$\hat{f}(x) = \underset{\alpha,\beta_1,\ldots,\beta_p}{\operatorname{argmin}} \sum_{i=1}^{n} K\left(\frac{x - x_i}{h}\right) \left(y_i - \alpha - \sum_{j=1}^{p} \beta_j (x - x_i)^j\right)^2$$

Nuevamente la prediccion en x es $\hat{\alpha}$.

```r
fit_1 <- loess(dist~speed, degree=1, data=cars)
fit_2 <- loess(dist~speed, degree=2, data=cars)
x_g_c <- seq(min(cars$speed), max(cars$speed), by=0.5)
plot(cars$speed, cars$dist, pch=16, col="grey50",
     xlab="Velocidad (mph)", ylab="Distancia (ft)",
     main="LOESS grado 1 vs. grado 2")
lines(x_g_c,predict(fit_1, data.frame(speed=x_g_c)),
      col="red", lwd=2)
lines(x_g_c,predict(fit_2, data.frame(speed=x_g_c)),
      col="blue", lwd=2)
legend("topleft", legend=c("Grado 1","Grado 2"),
       col=c("red","blue"), lwd=2, cex=0.8)
```



Comparacion: grados 0, 1 y 2



Comparacion con suavizamiento spline

Aspecto	Suaviz. spline	Regresion local (LOESS)
Estimacion	Global, penalizada	Local, ponderada por kernel
Hiperparametro	λ (rugosidad)	h (ancho de ventana)
Seleccion	LOOCV / GCV / REML	CV, regla del pulgar
Extremos	Lineal (spline natural)	Depende del grado
df	$df_\lambda = \mathrm{tr}(S)$	Vecinos efectivos

Ambos son **no parametricos**: no asumen ninguna forma funcional para f.

Puente: de los splines a los kernels

Repaso: funciones de densidad y distribucion

Proceso aleatorio generador de datos

Estimacion kernel de la densidad (KDE)

Kernel bivariado: hacia dimensiones mayores

Metodos kernel: regresion local

El kernel como medida de similitud

Sintesis y cierre

Referencias

El kernel como medida de similitud

Datos estructurados vs. no estructurados

Hasta ahora $\mathbf{x} \in \mathbb{R}^p$: la distancia $||\mathbf{x} - \mathbf{x}_i||$ tiene sentido.

Datos no estructurados: objetos sin coordenadas numericas:

- **Texto:** "El bus de la ruta 3 siempre llega tarde."
- **Secuencias:** ATGCTAGCTAGCTA...
- **Imagenes:** matrices de pixeles de distintas dimensiones.
- **Grafos:** red de calles y conexiones entre paradas.

En todos estos casos, no hay norma euclidiana. Pero si podemos definir **cuanto se parecen dos objetos**: eso es un kernel.

El espacio de Hilbert: intuicion

Para entender el kernel necesitamos la idea de **espacio de Hilbert**. Partamos desde lo familiar:

$\mathbb{R}^p$: cada punto tiene p coordenadas y el **producto interno** es $\langle \mathbf{u}, \mathbf{v} \rangle = \sum_{j=1}^{p} u_j v_j$. Este numero mide cuanto "apuntan en la misma direccion" $\mathbf{u}$ y $\mathbf{v}$.

Un **espacio de Hilbert** $\mathcal{H}$ generaliza esto:

- Puede tener **infinitas** dimensiones.
- Sigue teniendo un producto interno bien definido $\langle \cdot, \cdot \rangle_{\mathcal{H}}$.

Ejemplo concreto: el espacio de todas las funciones continuas en $[0, 1]$ con $\langle f, g \rangle = \int_0^1 f(x)g(x)\,dx$ es un espacio de Hilbert de dimension infinita. Cada "punto" es una funcion entera.

Puente: de los splines a los kernels

Repaso: funciones de densidad y distribucion

Proceso aleatorio generador de datos

Estimacion kernel de la densidad (KDE)

Kernel bivariado: hacia dimensiones mayores

Metodos kernel: regresion local

El kernel como medida de similitud

Sintesis y cierre

Referencias

El kernel de Mercer: definicion

Dado un espacio de objetos $\mathcal{X}$ (pueden ser textos, imagenes, vectores, lo que sea), un **kernel de Mercer** es una funcion:

$$K : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$$

tal que existe un espacio de Hilbert $\mathcal{H}$ y un mapa $\phi : \mathcal{X} \to \mathcal{H}$ con:

$$K(x, x') = \langle \phi(x), \phi(x') \rangle_{\mathcal{H}}$$

En palabras: $K(x, x')$ mide cuanto se parecen x y x', calculando el producto interno de sus representaciones en el espacio $\mathcal{H}$.

El truco del kernel

El problema sin el truco: Si $\mathcal{H}$ tiene dimension D (posiblemente $D = \infty$), necesitariamos: - Calcular y almacenar $\phi(\mathbf{x}_i) \in \mathbb{R}^D$ para cada uno de los n datos. - Operar en dimension D: costoso.

Con el truco: Solo calculamos $K(\mathbf{x}_i, \mathbf{x}_j)$ para cada par de datos. Esto es una matriz $n \times n$, independientemente de D.

El ejemplo: tres frases sobre el transporte

Tenemos tres frases. Queremos medir cuanto se parecen sin asumir que son vectores numericos.

```
x1 <- "el bus llega tarde siempre"
x2 <- "el bus sale tarde a veces"
x3 <- "el metro llega puntual siempre"
```

Kernel elegido: numero de palabras distintas que comparten dos frases.

$$K(x, x') = \left| \text{palabras}(x) \cap \text{palabras}(x') \right|$$

Este numero mide similitud: dos frases que comparten muchas palabras son mas parecidas.

Paso 1: computar K directamente

```
K_texto <- function(a, b) {
  pa <- unique(strsplit(a, " ")[[1]])
  pb <- unique(strsplit(b, " ")[[1]])
  intersect(pa, pb)          # palabras compartidas
}

cat("K(x1, x2) =", length(K_texto(x1, x2)),
    "| palabras:", paste(K_texto(x1,x2), collapse=", "), "\n")
```

```
## K(x1, x2) = 3 | palabras: el, bus, tarde
```

```
cat("K(x1, x3) =", length(K_texto(x1, x3)),
    "| palabras:", paste(K_texto(x1,x3), collapse=", "), "\n")
```

```
## K(x1, x3) = 3 | palabras: el, llega, siempre
```

```
cat("K(x2, x3) =", length(K_texto(x2, x3)),
    "| palabras:", paste(K_texto(x2,x3), collapse=", "), "\n")
```

```
## K(x2, x3) = 1 | palabras: el
```

$x1$ y $x2$ comparten mas palabras (ambas sobre el bus). $x2$ y $x3$ son las menos parecidas. Todo esto **sin construir ningun vector**.

Paso 2: que seria phi explicitamente

El mapa ϕ convierte cada frase en un vector binario: una coordenada por cada palabra del vocabulario total.

```
textos    <- c(x1, x2, x3)
vocab     <- sort(unique(
  unlist(strsplit(textos, " "))))
cat("Vocabulario (dim =", length(vocab), "):\n")
```

```
## Vocabulario (dim = 10 ):
```

```
cat(vocab, "\n\n")
```

```
## a bus el llega metro puntual sale siempre tarde veces
```

```
phi <- function(txt) {
  palabras <- strsplit(txt, " ")[[1]]
  setNames(as.integer(vocab %in% palabras), vocab)
}

phi_x1 <- phi(x1)
phi_x1      # vector de 10 coordenadas para x1
```

```
##       a     bus      el   llega   metro puntual    sale siempre   tarde   veces
##       0       1       1       1       0       0       0       1       1       0
```

Con solo 3 frases cortas ya tenemos un espacio de 10 dimensiones. El vector tiene muchos ceros.

Paso 3: verificar que K = producto interno de phi

```r
phi_x2 <- phi(x2)
phi_x3 <- phi(x3)

# K(x1, x2) computado de dos formas
cat("--- K(x1, x2) ---\n")
```

```
## --- K(x1, x2) ---
```

```r
cat("Directo:         ", length(K_texto(x1, x2)), "\n")
```

```
## Directo:          3
```

```r
cat("Prod. interno:   ", sum(phi_x1 * phi_x2),    "\n\n")
```

```
## Prod. interno:    3
```

```r
# Mostrar que coordenadas contribuyen
contribucion <- phi_x1 * phi_x2
cat("Coordenadas activas:",
    names(contribucion[contribucion > 0]), "\n")
```

```
## Coordenadas activas: bus el tarde
```

El producto interno $\langle \phi(x_1), \phi(x_2) \rangle$ suma 1 por cada palabra compartida: es identico al conteo directo. Confirmado: $K(x, x') = \langle \phi(x), \phi(x') \rangle$.

Paso 4: la matriz de Gram (todo lo que necesita un algoritmo)

Con $n = 3$ frases, un algoritmo (SVM, Kernel PCA, Kernel Ridge) solo necesita la **matriz de Gram K**:

```
K_num <- function(a, b)
  length(K_texto(a, b))       # version numerica

# Matriz n x n de similitudes
n  <- length(textos)
Gram <- matrix(0, n, n)
for(i in 1:n) for(j in 1:n)
  Gram[i,j] <- K_num(textos[i], textos[j])

rownames(Gram) <- colnames(Gram) <-
  c("x1 (bus tarde)", "x2 (bus puntual)", "x3 (metro)")
Gram
```

```
##                  x1 (bus tarde) x2 (bus puntual) x3 (metro)
## x1 (bus tarde)                5                3          3
## x2 (bus puntual)              3                6          1
## x3 (metro)                    3                1          5
```

Esta matriz $n \times n$ es **todo lo que necesita** cualquier algoritmo kernel. No se necesita ϕ en ningun paso.

Paso 5: por que nunca necesitamos phi

```r
# Con este vocabulario: 10 dimensiones
cat("dim phi (3 frases, palabras):", length(vocab), "\n")
```

```
## dim phi (3 frases, palabras): 10
```

```r
# Con bigrams de 27 caracteres posibles
cat("dim phi (bigramas):          ", 27^2, "\n")
```

```
## dim phi (bigramas):          729
```

```r
# Con trigramas
cat("dim phi (trigramas):         ", 27^3, "\n")
```

```
## dim phi (trigramas):         19683
```

A medida que el kernel se vuelve mas expresivo, ϕ crece exponencialmente o se vuelve infinito.

El truco: como $K(x, x') = \langle \phi(x), \phi(x') \rangle$ y los algoritmos solo acceden a los datos a traves de productos internos, podemos reemplazar cada $\langle \phi(x_i), \phi(x_j) \rangle$ por $K(x_i, x_j)$ y **nunca calcular** ϕ.

Ejemplo 2: el kernel polinomial en detalle

Para $\mathbf{x} = (x_1, x_2) \in \mathbb{R}^2$, el kernel polinomial de grado 2 es:

$$K(\mathbf{x}, \mathbf{x}') = (\mathbf{x} \cdot \mathbf{x}' + 1)^2$$

Expandiendo algebraicamente:

$$= (x_1 x_1' + x_2 x_2' + 1)^2 = x_1^2 x_1'^2 + x_2^2 x_2'^2 + 2x_1 x_2 x_1' x_2' + 2x_1 x_1' + 2x_2 x_2' + 1$$

Esto es el producto interno en $\mathbb{R}^6$ con:

$$\phi(\mathbf{x}) = (x_1^2,\ x_2^2,\ \sqrt{2}x_1 x_2,\ \sqrt{2}x_1,\ \sqrt{2}x_2,\ 1)$$

El kernel evalua el producto interno en $\mathbb{R}^6$ con **una sola operacion aritmetica**, sin construir ϕ.

Ejemplos de kernels

Sobre $\mathbb{R}^p$:

- **Lineal:** $K(\mathbf{x}, \mathbf{x}') = \mathbf{x} \cdot \mathbf{x}'$ (equivale a modelo lineal).
- **Polinomial grado d:** $K(\mathbf{x}, \mathbf{x}') = (\mathbf{x} \cdot \mathbf{x}' + c)^d$. Captura interacciones hasta grado d.
- **RBF (gaussiano):** $K(\mathbf{x}, \mathbf{x}') = e^{-||\mathbf{x}-\mathbf{x}'||^2/(2h^2)}$. Dimension infinita. El mas usado en la practica.

Sobre objetos no estructurados:

- **n-gramas (texto):** cuenta subsecuencias de longitud n comunes entre dos documentos.
- **Caminos en grafos:** cuenta caminos de longitud fija comunes entre dos redes.
- **Via embedding:** $K(x, x') = \langle \phi(x), \phi(x') \rangle$, donde ϕ es un modelo de lenguaje (word2vec, BERT) que convierte texto a vector numerico.

Aplicaciones del kernel: para que sirve

Una vez definido K, se dispone de un conjunto rico de herramientas. En todos los casos la clave es la misma: **el modelo aprende en el espacio de caracteristicas** $\mathcal{H}$ sin calcularlo explicitamente.

1. Clasificacion no lineal (SVM con kernel): Se busca el hiperplano de maxima margen en $\mathcal{H}$. En el espacio original $\mathcal{X}$, esto equivale a una frontera de decision **curvada y adaptada** a los datos. Util cuando las clases no son linealmente separables en $\mathcal{X}$.

Por ejemplo: separar correos spam de correos validos con un kernel de n-gramas sobre el texto.

Puente: de los splines a los kernels

Repaso: funciones de densidad y distribucion

Proceso aleatorio generador de datos

Estimacion kernel de la densidad (KDE)

Kernel bivariado: hacia dimensiones mayores

Metodos kernel: regresion local

El kernel como medida de similitud

Sintesis y cierre

Referencias

Aplicaciones del kernel: para que sirve (cont.)

2. Regresion no lineal (Kernel Ridge Regression): Extension de Ridge donde el modelo f se busca en $\mathcal{H}$ en lugar de en $\mathbb{R}^p$. La solucion incorpora no-linealidad sin especificar la forma de f: se define implicitamente por $f(x) = \sum_i \alpha_i K(x, x_i)$.

Util cuando la relacion entre X e Y es compleja y no se quiere elegir manualmente su forma funcional.

3. Reduccion de dimension no lineal (Kernel PCA): PCA clasico diagonaliza la matriz de covarianzas de X. Kernel PCA diagonaliza la matriz de covarianzas en $\mathcal{H}$, encontrando **direcciones principales no lineales**. Captura estructura en los datos que el PCA lineal no puede ver.

Por ejemplo: descubrir que datos en espiral en $\mathbb{R}^2$ son linealmente separables en un espacio de mayor dimension.

Aplicaciones del kernel: para que sirve (cont.)

4. Comparacion de distribuciones (MMD): Maximum Mean Discrepancy mide si dos conjuntos de datos provienen de la misma distribucion. La idea: si $f_1 = f_2$, el promedio de $K(x_i, \cdot)$ sobre las dos muestras deberia ser igual.

Util para detectar cambio de distribucion en datos de produccion (data drift): ¿los datos de hoy se parecen a los del entrenamiento?

5. Procesos gaussianos: Modelos probabilisticos de prediccion donde el kernel define la **covarianza entre predicciones** en puntos distintos: $\mathrm{Cov}(f(x), f(x')) = K(x, x')$.

Permiten no solo predecir $\hat{f}(x)$, sino tambien cuantificar la **incertidumbre** de la prediccion. Muy usados en optimizacion bayesiana (buscar el maximo de una funcion costosa con pocas evaluaciones).

Puente: de los splines a los kernels

Repaso: funciones de densidad y distribucion

Proceso aleatorio generador de datos

Estimacion kernel de la densidad (KDE)

Kernel bivariado: hacia dimensiones mayores

Metodos kernel: regresion local

El kernel como medida de similitud

Sintesis y cierre

Referencias

Sintesis y cierre

Mapa conceptual de la sesion

Metodo	Objetivo	Hiperparam.	Suave	Tipo datos
Histograma	f(x)	bins	No	1D
KDE (rect.)	f(x)	h	No	1D
KDE (gauss.)	f(x)	h	Si	1D
N-W (grado 0)	E(Y|x)	h	Si	1D
LOESS grado 1	E(Y|x)	h,span	Si	1D
LOESS grado 2	E(Y|x)	h,span	Si	1D
KDE 2D	f(x1,x2)	h	Si	2D
Kernel Mercer	Similitud	tipo K	–	Cualquier X

Hilo conductor del curso

Sesion 1: MCO – multicolinealidad – Ridge y Lasso.

Sesion 2: Validacion – ECM_{Test} – CV y Bootstrap.

Sesion 3: No-linealidad – Splines y GAMs.

Sesion 4: Estimacion local – KDE y regresion kernel, y similitud general con kernels de Mercer.

Los cuatro temas comparten: limitacion del modelo estandar, extension flexible, hiperparametro de complejidad, seleccion por CV.

Guia practica: que metodo usar

Situacion	Metodo
Estimar densidad univariada	KDE gaussiano (`density()`)
Estimar densidad bivariada	`kde2d()` con kernel gaussiano
Regresion, 1 variable continua	LOESS grado 1 o 2 (`loess()`)
Regresion, multiples variables	LOESS o GAM (Sesion 3)
Similitud entre puntos en $\mathbb{R}^p$	Kernel RBF o polinomial
Similitud entre textos o secuencias	Kernel de n-gramas o espectro
Texto o imagen via modelos modernos	Embedding + kernel gaussiano
Clasificacion no lineal	SVM con kernel RBF
Regresion no lineal sin elegir forma	Kernel Ridge Regression
Dimension no lineal	Kernel PCA
Detectar cambio de distribucion en datos	MMD con kernel RBF
Prediccion con incertidumbre cuantificada	Proceso gaussiano

Puente: de los splines a los kernels

Repaso: funciones de densidad y distribucion

Proceso aleatorio generador de datos

Estimacion kernel de la densidad (KDE)

Kernel bivariado: hacia dimensiones mayores

Metodos kernel: regresion local

El kernel como medida de similitud

Sintesis y cierre

Referencias

Referencias

Bibliografia

(1) Hastie, T., Tibshirani, R., Friedman, J. *The Elements of Statistical Learning*. Springer. (Cap. 6)

(2) James, G., Witten, D., Hastie, T., Tibshirani, R. *An Introduction to Statistical Learning with R*, 2a ed. Springer. (Cap. 7)

(3) Silverman, B. W. *Density Estimation for Statistics and Data Analysis*. Chapman and Hall, 1986.

(4) Scholkopf, B., Smola, A. *Learning with Kernels*. MIT Press, 2002.

Paquetes en R:

- `stats::density()` – KDE 1D con multiples kernels.
- `MASS::kde2d()` – KDE bivariado con kernel gaussiano.
- `stats::ksmooth()` – Nadaraya-Watson.
- `stats::loess()` – Regresion polinomial local.
- `kernlab` – Kernels de Mercer, SVM, Kernel PCA.